

算法 (AI) 第九周优秀作业

第一题

1. 用栈实现 dfs:

递归是函数对自身的不断调用, 在调用过程中, 相当于把当前递归层的局部变量保存在当前栈中, 再将下一层递归的变量入栈; 下一层退出时从栈中读取这些变量.....

当递归层操作全部完成, 就出栈, 返回了返回值.

把根结点压入堆栈 $\rightarrow$ pop一个结点 $\rightarrow$ 是目标 $\rightarrow$ 结束, 回传
不是 $\rightarrow$ 将尚未检验过的直接子节点加入栈中

```
stack.push(root)
while (!stack.isEmpty())
    node = stack.pop()
    for each neighbor node 相邻节点
        if neighbor node 未被访问
            stack.push(neighbor)
            stack.parent = node
    ...
```

第二题

2. $\text{prim}(MGraph G, \text{VertexType } u, p)$

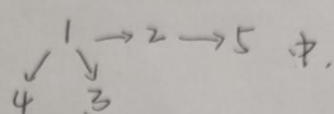
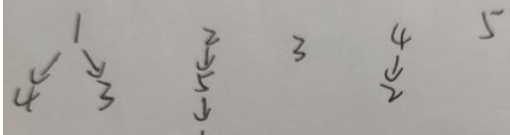
```
{
    k = LocateVex(G, u)
    for j=0 in G.vexnum do // 辅助数组初始化
        if (j != k)
            closedge[j] = {u, G.arcs[k][j].adj}

    closedge[k].lowcost = 0 // 初始化 u = {u}
    for i=1 in G.vexnum do
        { k = minnum(closedge)
          print(closedge[k].adjvex, G.vex[k]) // 输出生成
          closedge[k].lowcost = 0 // k 并入 u 集 树的所有边
          for j=0 in G.vexnum do
              if (G.arcs[k][j].adj < closedge[j].lowcost)
                  closedge[j] = {G.vexs[k], G.arcs[k][j].adj}
          }

        closedge[p].lowcost = 0
        k = maxnum(closedge[p~Q])
        delete(closedge[k].adjvex, G.vexs[k])
    }
}
```

思想: 把修改的边加入到最小生成树中, 在一条含有修改边的回路去掉权值最大即可

第三题

在有向图：中，
按深度优先可得到森林：

可知 1, 2 间存在一条 1 到 2 的路径，
且 $d[1] < d[2]$ ，但 2 不是 1 的子节点。
因此猜想不成立。

第四题

Prim 算法的时间复杂度为 $O(|E| \lg |V|)$ ，Kruskal 算法的时间复杂度为 $O(|E| \lg |E|)$ ，要使 Prim 算法慢于 Kruskal 算法，即要求 $|E| \lg |V| > |E| \lg |E|$ ，可得 $V > E$ 。又因为这两个算法是用来求图的最小生成树，所以 $E \geq V - 1$ 。综上所述，当 $E = V - 1$ 时，Prim 算法要慢于 Kruskal 算法，故存在这样的图。